

Security Proofs for Privately Verifiable BBS-Based Anonymous Credentials

JONATHAN KATZ*

MARIANA RAYKOVA*

March 20, 2025

Abstract

Publicly verifiable BBS-based anonymous credentials have been studied in several works, most recently by Tessaro and Zhu. We provide self-contained proofs of security for a *privately* verifiable version of BBS-based anonymous credentials. We first prove unforgeability of BBS itself as a message authentication code, and then prove security of the interactive proof-of-possession of a credential.

1 BBS-Based Anonymous Credentials

BBS signatures trace their roots to work of Boneh, Boyen, and Shacham [3], and were explicitly proposed for anonymous credentials by Camenisch and Lysyanskaya [5]. Camenisch et al. [4] showed (honest-verifier) zero-knowledge (ZK) proofs for partial showings of credentials, based on a variant of the original scheme called BBS^+ [1]. Tessaro and Zhu [10] recently showed that the original BBS scheme can be proven secure, and the scheme is now being proposed as a draft standard [7].

The aforementioned works all consider a *publicly* verifiable version of scheme where anyone can verify credentials issued by the issuer. We describe a *privately* verifiable version of the scheme [2, 9] where only the issuer can verify credentials using the secret key it holds. (This is analogous to the distinction between a signature and a message authentication code.) An advantage of the privately verifiable version is that it does not require pairings.

2 The BBS-MAC

2.1 Scheme Definition

Fix a group $\mathbb{G}$ of prime order p , and a parameter ℓ . Uniform generators $g, h_1, \dots, h_\ell \leftarrow \mathbb{G} \setminus \{1\}$ are chosen; these could either be shared public parameters, or chosen by an issuer as part of key generation. The privately verifiable version of BBS (i.e., the BBS-MAC) works as follows:

- To generate a secret key, choose $x \leftarrow \mathbb{Z}_p^*$.
- To authenticate $\mathbf{m} = (m_1, \dots, m_\ell) \in \mathbb{Z}_p^\ell$ with secret key x , the issuer chooses $e \leftarrow \mathbb{Z}_p \setminus \{-x\}$ and outputs the tag $\left(\left(g \cdot \prod_{i=1}^\ell h_i^{m_i} \right)^{1/(e+x)}, e \right)$. We denote this process by $\text{Mac}_x(\mathbf{m})$.

*Google.

- To verify $\text{tag}(A, e)$ on $\mathbf{m} = (m_1, \dots, m_\ell) \in \mathbb{Z}_p^\ell$ using secret key x , the issuer checks if

$$A^{e+x} \stackrel{?}{=} g \cdot \prod_{i=1}^{\ell} h_i^{m_i}.$$

We use the standard notion of strong unforgeability [6, Def. 4.3], given next for concreteness.

Definition 1. For an algorithm $\mathcal{A}$, define

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{BBS}} \\ = \Pr \left[\begin{array}{l} g, h_1, \dots, h_\ell \leftarrow \mathbb{G} \setminus \{1\}; x \leftarrow \mathbb{Z}_p^*; \\ (\mathbf{m}^*, (A^*, e^*)) \leftarrow \mathcal{A}^{\text{Mac}_x(\cdot)}(g, \{h_i\}, g^x) : (A^*)^{e^*+x} = g \cdot \prod_{i=1}^{\ell} h_i^{m_i^*} \wedge (\mathbf{m}^*, (A^*, e^*)) \notin Q \end{array} \right], \end{aligned}$$

where Q represents the set of queries/responses from $\text{Mac}_x(\cdot)$.

Note that in some applications, the associated value $w \stackrel{\text{def}}{=} g^x$ may be public (even though this still does not allow the tag to be verified by anyone except the issuer); for this reason, we give w to the adversary in the security definition.

2.2 Proof of Security

We rely on hardness of the discrete-logarithm and q -strong Diffie-Hellman (q -SDH) problems:

Definition 2. For an algorithm $\mathcal{A}$, define

$$\text{Adv}_{\mathcal{A}}^{\text{dlog}} = \Pr[g \leftarrow \mathbb{G} \setminus \{1\}; x \leftarrow \mathbb{Z}_p^* : \mathcal{A}(g, g^x) = x].$$

Definition 3. For an algorithm $\mathcal{A}$, define

$$\text{Adv}_{\mathcal{A}}^{q\text{-SDH}} = \Pr \left[g \leftarrow \mathbb{G} \setminus \{1\}; x \leftarrow \mathbb{Z}_p^*; (c, Z) \leftarrow \mathcal{A} \left(\{g^{x^i}\}_{i=0}^q \right) : Z = g^{1/(x+c)} \right].$$

In our analysis, we rely on the following standard results.

Remark 1. Given $\{X_i = g^{x^i}\}_{i=0}^q$, it is possible to compute $g^{P(x)}$ for any polynomial $P(X) \in \mathbb{Z}_p[X]$ of degree at most q . Specifically, if $P(X) = \sum_{i=0}^q p_i X^i$, then $g^{P(x)} = \prod_{i=0}^q X_i^{p_i}$.

Remark 2. There is an efficient algorithm *Solve* that takes as input $\{g^{x^i}\}_{i=0}^q$, $e \neq -x$, a polynomial P of degree at most $q+1$ with $P(-e) \neq 0$, and $A = g^{P(x)/(x+e)}$, and outputs $g^{1/(x+e)}$. The algorithm works by writing $P(X) = P'(X) \cdot (X+e) + r$ with $r \in \mathbb{Z}_p^*$; then $A = g^{P'(x)} \cdot g^{r/(x+e)}$ and so $(A \cdot g^{-P'(x)})^{1/r} = g^{1/(x+e)}$. (Note $g^{-P'(x)}$ can be computed using the previous remark.)

Theorem 1. For every adversary $\mathcal{A}$ making q_{mac} MAC queries, there are algorithms $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ whose running times are $\text{poly}(\ell, q_{\text{mac}})$ more than $\mathcal{A}$, such that

$$\text{Adv}_{\mathcal{A}}^{\text{BBS}} \leq q_{\text{mac}} \cdot \text{Adv}_{\mathcal{B}_1}^{q_{\text{mac}}\text{-SDH}} + \text{Adv}_{\mathcal{B}_2}^{\text{dlog}} + \text{Adv}_{\mathcal{B}_3}^{q_{\text{mac}}\text{-SDH}} + (q_{\text{mac}}^2 + 1)/p.$$

Proof. Our proof follows exactly along the lines of the proof by Tessaro and Zhu [10] in the publicly verifiable setting; we highlight the few differences. For an adversary $\mathcal{A}$, let $\mathbf{m}_i$ be the i th vector whose tag is requested by $\mathcal{A}$, and let (A_i, e_i) be the corresponding tag returned. We assume without loss of generality that $\mathcal{A}$ always requests tags on exactly q_{mac} vectors. At the end of its execution, $\mathcal{A}$ outputs $\mathbf{m}^*$ and (A^*, e^*) , and succeeds if (1) $(A^*)^{e^*+x} = g \cdot \prod h_i^{m_i^*}$ and (2) $\mathcal{A}$ never previously received tag (A^*, e^*) on vector $\mathbf{m}^*$. We let Succ be the event that $\mathcal{A}$ succeeds, so $\text{Adv}_{\mathcal{A}}^{\text{BBS}} = \Pr[\text{Succ}]$.

In an execution of $\mathcal{A}$, we distinguish three cases. Let Forge_1 be the event that there is an index i such that $e^* = e_i$ but $A^* \neq A_i$ and $\mathbf{m}^* \neq \mathbf{m}_i$; let Forge_2 be the event that $A^* = 1$, or there is an index i such that $e^* = e_i$ and $A^* = A_i$ but $\mathbf{m}^* \neq \mathbf{m}_i$ (note that $e^* = e_i$, $\mathbf{m}^* = \mathbf{m}_i$, and $A^* \neq A_i$ cannot occur because then (A^*, e^*) is not a valid tag for $\mathbf{m}^*$); and let Forge_3 be the event that $e^* \notin \{e_1, \dots, e_{q_{\text{mac}}}\}$ and $A^* \neq 1$. In the three claims that follow, we upper bound the probabilities of $\text{Succ} \wedge \text{Forge}_i$ for $i = 1, 2, 3$.

Claim 1. *There is an algorithm $\mathcal{B}_1$, whose running time is $\text{poly}(\ell, q_{\text{mac}})$ more than the running time of $\mathcal{A}$, such that $\frac{1}{q_{\text{mac}}} \cdot \Pr[\text{Succ} \wedge \text{Forge}_1] - q_{\text{mac}}/p \leq \text{Adv}_{\mathcal{B}_1}^{q_{\text{mac}}\text{-SDH}}$.*

Proof. $\mathcal{B}_1$ is given as input $\{X_i = g^{x^i}\}_{i=0}^{q_{\text{mac}}}$. It sets $g := X_0$ and does:

- Choose $i^* \leftarrow \{1, \dots, q_{\text{mac}}\}$, $\epsilon_1, \dots, \epsilon_{q_{\text{mac}}} \leftarrow \mathbb{Z}_p$, and $\alpha, \beta_1, \dots, \beta_\ell \leftarrow \mathbb{Z}_p^*$. Define the polynomial $P(X) = \prod_{i \neq i^*} (X + \epsilon_i)$.
- Set $\hat{g} := g^{\alpha \cdot P(x) \cdot (x + \epsilon_{i^*})}$, $h_i := g^{\beta_i \cdot P(x)}$ for $i = 1, \dots, \ell$, and $w := X_1 = g^x$.
- Run $\mathcal{A}(\hat{g}, \{h_i\}, w)$. When $\mathcal{A}$ makes its i th request for a tag on the vector $\mathbf{m} = (m_1, \dots, m_\ell)$, there are two sub-cases:

- For $i \neq i^*$, let $P'(X) = P(X)/(X + \epsilon_i)$ and return the tag $(A_i, e_i = \epsilon_i)$ where

$$A_i := g^{P'(x) \cdot (\alpha(x + \epsilon_{i^*}) + \sum_i \beta_i m_i)} = g^{\alpha \cdot P'(x) \cdot (x + \epsilon_{i^*})} \cdot \prod g^{m_i \cdot \beta_i \cdot P'(x)}.$$

- For $i = i^*$, set $A_{i^*} := g^{\alpha \cdot P(x)}$ and $e_{i^*} := \epsilon_{i^*} + \alpha^{-1} \cdot \sum_{i=1}^\ell \beta_i m_i$, and return (A_{i^*}, e_{i^*}) . (Also, for the analysis below, define $B_{i^*} = \hat{g} \cdot \prod_i h_i^{m_i}$.)

- When $\mathcal{A}$ outputs $(\mathbf{m}^*, (A^*, e^*))$, if $e^* = e_{i^*}$, $A^* \neq A_{i^*}$, $\mathbf{m}^* \neq \mathbf{m}_i$, and $e_{i^*} \notin \{\epsilon_i\}_{i \neq i^*}$ then continue; otherwise halt. Let $\mathbf{m} = \mathbf{m}_i$. Set $\gamma := \sum_i \beta_i \cdot (m_i^* - m_i)$ and compute $A := (A^* \cdot A_{i^*}^{-1})^{1/\gamma}$. Then let

$$Z := \text{Solve}(\{X_i\}, e_{i^*}, P(X), A),$$

and output (Z, e_{i^*}) .

Note that if $P(x) \cdot (x + \epsilon_{i^*}) = 0$ then $-x \in \{\epsilon_1, \dots, \epsilon_{q_{\text{mac}}}\}$ and $\mathcal{B}_1$ can trivially solve the $q_{\text{mac}}\text{-SDH}$ problem. Similarly, if $B_{i^*} = 1$ then (letting $\mathbf{m} = \mathbf{m}_{i^*}$)

$$\begin{aligned} 0 &= \alpha \cdot P(x) \cdot (x + \epsilon_{i^*}) + \sum_i \beta_i m_i P(x) = \alpha \cdot P(x) \cdot \left(x + \epsilon_{i^*} + \alpha^{-1} \cdot \sum_i \beta_i m_i \right) \\ &= \alpha \cdot P(x) \cdot (x + e_{i^*}), \end{aligned}$$

so either $P(x) = 0$ or $x = -e_{i^*}$ and $\mathcal{B}_1$ can again trivially solve the $q_{\text{mac}}\text{-SDH}$ problem. We condition on non-occurrence of these events in the analysis that follows.

The inputs to $\mathcal{A}$ are distributed as in the experiment defining strong unforgeability of BBS-MAC. For the tags $\{(A_i, e_i)\}$, note first that the $\{e_i\}$ are all distributed exactly as in the BBS-MAC experiment. The $\{A_i\}_{i \neq i^*}$ also clearly have the correct distribution. As for A_{i^*} , we have

$$(A_{i^*})^{x+e_{i^*}} = g^{\alpha P(x)(x+e_{i^*})} \cdot g^{\sum_i \beta_i P(x)m_i} = \hat{g} \cdot \prod_i h_i^{m_i},$$

and so A_{i^*} is also correctly distributed.

Assume Forge_1 and Succ occur. (In contrast to the publicly verifiable setting, $\mathcal{B}_1$ cannot tell whether Succ occurs. But this does not matter.) Then with probability at least $1/q_{\text{mac}}$ it holds that $e^* = e_{i^*}$, $A^* \neq A_{i^*}$, and $\mathbf{m}^* \neq \mathbf{m}_i = \mathbf{m}$. Moreover, $e_{i^*} \notin \{e_i\}_{i \neq i^*}$ except with probability at most $(q_{\text{mac}} - 1)/(p - 1) < q_{\text{mac}}/p$. Since Succ occurs, we have

$$A^* A_{i^*}^{-1} = \left(\prod_i h_i^{(m_i^* - m_i)} \right)^{1/(x+e_{i^*})} = g^{P(x) \cdot \sum_i \beta_i (m_i^* - m_i)/(x+e_{i^*})}.$$

Note that $\gamma = \sum_i \beta_i (m_i^* - m_i) \neq 0$ since $A^* A_{i^*}^{-1} \neq 1$. Then $A = (A^* A_{i^*}^{-1})^{1/\gamma} = g^{P(x)/(x+e_{i^*})}$, and so $\mathcal{B}_1$ solves the q_{mac} -SDH problem. The claim follows. $\square$

Claim 2. *There is an algorithm $\mathcal{B}_2$, whose running time is $\text{poly}(\ell)$ more than the running time of $\mathcal{A}$, such that $\Pr[\text{Succ} \wedge \text{Forge}_2] - 1/p \leq \text{Adv}_{\mathcal{B}_2}^{\text{dlog}}$.*

Proof. $\mathcal{B}_2$ is given as input g and $h = g^x$. It then does the following:

- For $i = 1, \dots, \ell$, choose $\alpha_i, \beta_i \leftarrow \mathbb{Z}_p$ and set $h_i := g^{\beta_i} h^{-\alpha_i}$. (If $h_i = 1$, resample until $h_i \neq 1$.)
- Choose $\bar{x} \leftarrow \mathbb{Z}_p$ and set $w := g^{\bar{x}}$. Run $\mathcal{A}(g, \{h_i\}, w)$. When $\mathcal{A}$ requests a tag, answer the request honestly (using $\bar{x}$).
- When $\mathcal{A}$ outputs $(\mathbf{m}^*, (A^*, e^*))$, if Forge_2 does not occur then halt. Otherwise:
 1. If $A^* = 1$, set $u := 1$ and $\mathbf{v} := \mathbf{m}^*$.
 2. Otherwise, let i be such that $e^* = e_i$ and $A^* = A_i$ but $\mathbf{m} \neq \mathbf{m}_i$; then set $u := 0$ and $\mathbf{v} := \mathbf{m}^* - \mathbf{m}_i$.

Let $\mathbf{v} = (v_1, \dots, v_\ell)$. If $\sum_i v_i \alpha_i = 0$, halt. Otherwise, output $(u + \sum_i v_i \beta_i) \cdot (\sum_i v_i \alpha_i)^{-1} \bmod p$.

Each h_i is uniformly distributed in $\mathbb{G} \setminus \{1\}$ and so the inputs to $\mathcal{A}$ are distributed as in the experiment defining strong unforgeability of BBS-MAC. Tags returned to $\mathcal{A}$ obviously have the correct distribution. Finally, assume Succ and Forge_2 occur. (In contrast to the publicly verifiable setting, $\mathcal{B}_2$ cannot tell whether Succ occurs. But this does not matter.) Then $(A^*)^{e^* + \bar{x}} = g \cdot \prod_j h_j^{m_j^*}$. If $A^* = 1$ then $g \cdot \prod_j h_j^{m_j^*} = 1$; otherwise,

$$g \cdot \prod_j h_j^{m_j^*} = (A^*)^{e^* + \bar{x}} = (A_i)^{e_i + \bar{x}} = g \cdot \prod_j h_j^{m_{i,j}}$$

and so $\prod_j h_j^{m_j^* - m_{i,j}} = 1$. Either way, we have $g^{u + \sum_i v_i \beta_i} h^{-\sum_i v_i \alpha_i} = 1$, and so $\mathcal{B}_2$ correctly solves the discrete-logarithm problem as long as $\sum_i v_i \alpha_i \neq 0$. A standard argument (using the fact that the $\{\alpha_i\}$ are uniform and independently distributed in $\mathbb{Z}_p$ conditioned on the view of $\mathcal{A}$) shows that the probability that $\sum_i v_i \alpha_i = 0$ is $1/p$. This concludes the proof. $\square$

Claim 3. *There is an algorithm $\mathcal{B}_3$, whose running time is $\text{poly}(\ell, q_{\text{mac}})$ more than the running time of $\mathcal{A}$, such that $\Pr[\text{Succ} \wedge \text{Forge}_3] \leq \text{Adv}_{\mathcal{B}_3}^{q_{\text{mac}}\text{-SDH}}$.*

Proof. $\mathcal{B}_3$ is given as input $\{X_i = g^{x^i}\}_{i=0}^{q_{\text{mac}}}$. It sets $g := X_0$ and does:

- Choose $\alpha, \beta_1, \dots, \beta_\ell \leftarrow \mathbb{Z}_p^*$ and $e_1, \dots, e_{q_{\text{mac}}} \leftarrow \mathbb{Z}_p$. Define $P(X) \stackrel{\text{def}}{=} \prod_{i=1}^{q_{\text{mac}}} (X + e_i)$.
- Set $\hat{g} := g^{\alpha \cdot P(x)}$, $h_i := g^{\beta_i \cdot P(x)}$ for $i = 1, \dots, \ell$, and $w := X_1 = g^x$.
- Run $\mathcal{A}(\hat{g}, \{h_i\}, w)$. When $\mathcal{A}$ makes its i th request for a tag on the vector $\mathbf{m} = (m_1, \dots, m_\ell)$, let $P'(X) = P(X)/(X + e_i) = \prod_{j \neq i} (X + e_j)$ and return the tag (A_i, e_i) where

$$A_i := g^{\alpha \cdot P'(x)} \cdot \prod g^{m_i \cdot \beta_i \cdot P'(x)}.$$

- When $\mathcal{A}$ outputs $(\mathbf{m}^*, (A^*, e^*))$ with $\mathbf{m}^* = (m_1^*, \dots, m_\ell^*)$, if Forge_3 does not occur then halt. If $g^{-e^*} = X_1$ then output $(0, g^{1/-e^*})$. Otherwise, let

$$Z := \text{Solve} \left(\{X_i\}_{i=0}^q, e^*, (\alpha + \sum_i m_i^* \beta_i) \cdot P(X), A^* \right)$$

and output (e^*, Z) .

Note that if $P(x) = 0$ then $-x \in \{e_1, \dots, e_{q_{\text{mac}}}\}$ and $\mathcal{B}_3$ can trivially solve the q_{mac} -SDH problem. We therefore condition on $P(x) \neq 0$ in what follows. In that case $\hat{g}, h_1, \dots, h_\ell$ are uniformly distributed in $\mathbb{G} \setminus \{1\}$, and $w = g^x$ for a uniform $x \in \mathbb{Z}_p^*$; thus, the inputs to $\mathcal{A}$ are distributed as in the experiment defining strong unforgeability of BBS-MAC. Tags returned to $\mathcal{A}$ by $\mathcal{B}_3$ also have the correct distribution, since each e_i is uniform in $\mathbb{Z}_p \setminus \{-x\}$ and

$$A_i = g^{\alpha \cdot P'(x)} \cdot \prod g^{m_i \cdot \beta_i \cdot P'(x)} = g^{\alpha \cdot P(x)/(x+e_i)} \cdot \prod g^{m_i \cdot \beta_i \cdot P(x)/(x+e_i)} = \left(\hat{g} \cdot \prod h_i^{m_i} \right)^{1/(x+e_i)}.$$

Finally, assume Succ and Forge_3 occur. (Note that, in contrast to the publicly verifiable setting, $\mathcal{B}_3$ cannot tell whether Succ occurs. But this does not matter.) If $x + e^* = 0$ then $\mathcal{B}_3$ clearly succeeds. Otherwise,

$$A^* = \left(\hat{g} \cdot \prod h_i^{m_i^*} \right)^{1/(x+e^*)} = \left(g^\alpha \cdot \prod g^{\beta_i m_i^*} \right)^{P(x)/(x+e^*)} = g^{(\alpha + \sum_i m_i^* \beta_i) \cdot P(x)/(x+e^*)}$$

and so $Z = g^{1/(x+e^*)}$. (Note that $\alpha + \sum_i m_i^* \beta_i \neq 0$ since $A^* \neq 1$, and $P(-e^*) \neq 0$ because $e^* \notin \{e_1, \dots, e_{q_{\text{mac}}}\}$.) We conclude that $\text{Adv}_{\mathcal{B}_3}^{q_{\text{mac}}\text{-SDH}} \geq \Pr[\text{Succ} \wedge \text{Forge}_3]$. $\square$

This completes the proof. $\square$

3 Proof of Possession

It is possible to give an efficient proof of possession of a BBS tag on a particular vector (without revealing anything else about the signature) to the issuer who knows the private key x . In fact, even more is possible. Let $[\ell] = \{1, \dots, \ell\}$, and $\mathcal{D} \subseteq [\ell]$. Then one can prove to the issuer possession of a tag on a vector whose entries in $\mathcal{D}$ are equal to $\{m_i\}_{i \in \mathcal{D}}$, without revealing anything about the entries of the vector at indices not in $\mathcal{D}$. The protocol we show is adapted from the work of Camenisch et al. [4]. Tessaro and Zhu [10, Appendix A] show an alternate protocol that is slightly more efficient, but requires an additional assumption to be proven secure.

Assume a client has tag (A, e) on vector $(m_1, \dots, m_\ell)$, and let $\mathcal{D}$ be the indices whose entries will be disclosed. Let $B = g \cdot \prod_{i=1}^\ell h_i^{m_i}$. Protocol Π_{BBS} proceeds as follows:

1. The client chooses $r_1, r_2 \leftarrow \mathbb{Z}_p^*$ and sets $A' := A^{r_1 r_2}$, $\bar{B} := B^{r_1}$, and $r_3 := r_1^{-1}$.
2. It then sends $\{m_i\}_{i \in \mathcal{D}}$ and $A', \bar{B}$ along with a proof that there exist $\{m_i\}_{i \notin \mathcal{D}}, e, r_2, r_3$ such that¹ (1) $(A')^x = (A')^{-e} \cdot \bar{B}^{r_2}$ and (2) $H_1 \stackrel{\text{def}}{=} g \cdot \prod_{i \in \mathcal{D}} h_i^{m_i} = \bar{B}^{r_3} \cdot \prod_{i \notin \mathcal{D}} h_i^{-m_i}$. The proof is done as follows (we describe it as an interactive proof, but it can be made non-interactive using the Fiat-Shamir transform):
 - (a) The client chooses $\{m'_i\}_{i \notin \mathcal{D}}, e', r'_2, r'_3 \leftarrow \mathbb{Z}_p$, and sends to the issuer $A_1 := (A')^{e'} \cdot \bar{B}^{r'_2}$ and $A_2 := \bar{B}^{r'_3} \cdot \prod_{i \notin \mathcal{D}} h_i^{m'_i}$.
 - (b) The issuer chooses $c \leftarrow \mathbb{Z}_p$ and sends it to the client.
 - (c) The client sends $\bar{e} := -c \cdot e + e'$, $\bar{r}_2 := c \cdot r_2 + r'_2$, $\bar{r}_3 := c \cdot r_3 + r'_3$, and $\{\bar{m}_i := -c \cdot m_i + m'_i\}_{i \notin \mathcal{D}}$.
 - (d) The issuer computes $\bar{A} = (A')^x$ and $H_1 := g \cdot \prod_{i \in \mathcal{D}} h_i^{m_i}$, and then accepts iff $A' \neq 1$, $(A')^{\bar{e}} \bar{B}^{\bar{r}_2} \stackrel{?}{=} \bar{A}^c \cdot A_1$, and $\bar{B}^{\bar{r}_3} \prod_{i \notin \mathcal{D}} h_i^{\bar{m}_i} \stackrel{?}{=} H_1^c \cdot A_2$.

In the following sections, we show that Π_{BBS} satisfies *special soundness* and *witness indistinguishability*. Appropriate definitions are given below.

3.1 Special Soundness

Theorem 2. *There is an efficient algorithm Ext that, given two accepting transcripts of Π_{BBS} with the same initial message and different challenges, computes a valid tag on a vector consistent with the disclosed entries in the initial message. I.e., given initial message $\{m_i\}_{i \in \mathcal{D}}, A', \bar{B}, A_1, A_2$, distinct challenges c, c' , and corresponding accepting responses $\bar{e}, \bar{r}_2, \bar{r}_3, \{\bar{m}_i\}_{i \notin \mathcal{D}}$ and $e', r'_2, r'_3, \{m'_i\}_{i \notin \mathcal{D}}$, algorithm Ext outputs $\{m_i\}_{i \notin \mathcal{D}}$ and (A, e) such that $A^{x+e} = g \cdot \prod_i h_i^{m_i}$.*

Proof. We first show (using standard techniques) how to use the given information to compute $\{m_i\}_{i \notin \mathcal{D}}, e, r_2, r_3$ such that $(A')^x = (A')^{-e} \cdot \bar{B}^{r_2}$ and $g \cdot \prod_{i \in \mathcal{D}} h_i^{m_i} = \bar{B}^{r_3} \cdot \prod_{i \notin \mathcal{D}} h_i^{-m_i}$. We then show how that suffices to prove the theorem.

Let $\bar{A} = (A')^x$. (Note that Ext cannot directly compute $\bar{A}$, but prior knowledge of $\bar{A}$ will not be needed by Ext in what follows.) Since the two transcripts are accepting, we have

$$\begin{aligned} (A')^{\bar{e}} \bar{B}^{\bar{r}_2} / \bar{A}^c &= A_1 = (A')^{e'} \bar{B}^{r'_2} / \bar{A}^{c'} \\ &\Rightarrow (A')^{\delta_e} \bar{B}^{\delta_{r_2}} = \bar{A}^{\delta_c}, \end{aligned}$$

¹Note that the client cannot compute $(A')^x$. But the issuer can compute it when verifying the proof.

where $\delta_e = \bar{e} - \bar{e}'$, $\delta_{r_2} = \bar{r}_2 - \bar{r}_2'$, and $\delta_c = c - c' \neq 0$. Thus, Ext can compute $e = -\delta_e/\delta_c$ and $r_2 = \delta_{r_2}/\delta_c$ such that $\bar{A} = (A')^{-e} \cdot \bar{B}^{r_2}$. (In particular, this allows Ext to compute $\bar{A}$.) Letting $H_1 = g \cdot \prod_{i \in \mathcal{D}} h_i^{m_i}$, we similarly have

$$\begin{aligned} H_1^{-c} \bar{B}^{\bar{r}_3} \prod_{i \notin \mathcal{D}} h_i^{\bar{m}_i} &= A_2 = H_1^{-c'} \bar{B}^{\bar{r}_3'} \prod_{i \notin \mathcal{D}} h_i^{\bar{m}_i'} \\ &\Rightarrow \bar{B}^{\delta_{r_3}} \prod_{i \notin \mathcal{D}} h_i^{\delta_{m_i}} = H_1^{\delta_c}, \end{aligned}$$

where $\delta_{r_3} = \bar{r}_3 - \bar{r}_3'$, $\delta_{m_i} = \bar{m}_i - \bar{m}_i'$, and δ_c is as above. Thus, Ext can compute $r_3 = \delta_{r_3}/\delta_c$ and $\{m_i = -\delta_{m_i}/\delta_c\}_{i \notin \mathcal{D}}$ such that $H_1 = g \prod_{i \in \mathcal{D}} h_i^{m_i} = \bar{B}^{r_3} \prod_{i \notin \mathcal{D}} h_i^{-m_i}$.

If $r_2 = 0$ then (since $A' \neq 1$) $x = -e$ and Ext can compute a valid tag directly. If $r_2 \neq 0$ then

$$\begin{aligned} (A')^{(x+e)r_3/r_2} &= \bar{B}^{r_3} = g \prod_i h_i^{m_i} \\ &\Rightarrow (A')^{r_3/r_2} = \left(g \cdot \prod_{i \in [\ell]} h_i^{m_i} \right)^{1/(x+e)}, \end{aligned}$$

and so $((A')^{r_3/r_2}, e)$ is a valid tag on $(m_1, \dots, m_\ell)$. $\square$

Remark: In general, we would like to claim that Π_{BBS} is a *proof of knowledge* by showing an efficient knowledge extractor KE that, given access to a prover P^* who succeeds in Π_{BBS} with high probability, can (with high probability) compute a valid tag on a vector consistent with the disclosed entries in the first message sent by P^* . In the privately verifiable setting, we cannot directly use Theorem 2 to prove this because KE cannot tell when P^* succeeds. There are at least two ways to deal with this:

- We may assume KE has access to a “DDH oracle” that, given group elements (U, V) , returns 1 iff $U^x = V$. Given such an oracle, KE can tell when a proof is correct by checking that $((A')^c)^x = (A')^{\bar{e}} \bar{B}^{\bar{r}_2}/A_1$. We can then use standard techniques to ensure that if P^* succeeds with probability ϵ , then KE extracts a tag of the desired form with the same probability ϵ .

Note that in this case, we may need to require other assumptions to hold even in the presence of the DDH oracle.

- Alternately, we can just have KE run P^* twice (using two uniform challenges), assume P^* succeeded both times, and then run Ext from Theorem 2. Here, we pay the price in the extraction probability: if P^* succeeds with probability ϵ , then KE now extracts a tag of the desired form with probability only $\epsilon^2 - 1/p$.

3.2 Witness Indistinguishability

We now prove a strong form of witness indistinguishability (WI): even a cheating issuer cannot distinguish proofs of possession for two valid tags on two different vectors, as long as the disclosed entries of those vectors are the same.

Let P_{BBS} denote the honest prover algorithm in Π_{BBS} , and let $b' \leftarrow \langle P_{\text{BBS}}(\mathcal{D}, \bar{m}, A, e), \mathcal{A} \rangle$ denote an interaction of P_{BBS} (holding the indicated inputs) with possibly malicious verifier $\mathcal{A}$, after which $\mathcal{A}$ outputs b' .

Definition 4. For an adversary $\mathcal{A}$, define

$$\text{Adv}_{\mathcal{A}}^{\text{BBS-WI}} = \left| \Pr \left[\begin{array}{l} \left(\begin{array}{l} g, \{h_i\}, x, \mathcal{D}, \mathbf{m}^{(0)}, \mathbf{m}^{(1)}, \\ (A^{(0)}, e^{(0)}), (A^{(1)}, e^{(1)}) \end{array} \right) \leftarrow \mathcal{A}; \\ b \leftarrow \{0, 1\}; \\ b' \leftarrow \langle P_{\text{BBS}}(\mathcal{D}, \mathbf{m}^{(b)}, A^{(b)}, e^{(b)}), \mathcal{A} \rangle \end{array} : \begin{array}{l} (A^{(0)})^{x+e^{(0)}} = g \prod_i h_i^{m_i^{(0)}} \neq 1 \\ \wedge (A^{(1)})^{x+e^{(1)}} = g \prod_i h_i^{m_i^{(1)}} \neq 1 \\ \wedge \{m_i^{(0)}\}_{i \in \mathcal{D}} = \{m_i^{(1)}\}_{i \in \mathcal{D}} \\ \wedge b = b' \end{array} \right] \right| - \frac{1}{2}.$$

Remark: In the above definition, we must force $\mathcal{A}$ to output two valid tags relative the same key x ; otherwise, it is trivial for $\mathcal{A}$ to distinguish which tag is being used. Since clients cannot verify correctness of tags on their own, this means that a potentially malicious issuer should prove correctness of the tags they issue.

We also exclude the case of a tag on a message $\mathbf{m}$ for which $g \cdot \prod_i h_i^{m_i} = 1$. If parameters are chosen at random, the probability of such a message ever being used is negligible. If parameters are chosen by the issuer, the client should check this condition before accepting a tag.

Theorem 3. For any (even unbounded) adversary $\mathcal{A}$, we have $\text{Adv}_{\mathcal{A}}^{\text{BBS-WI}} = 0$.

Proof. Fix $g, \{h_i\}, x, \mathcal{D}, \mathbf{m}^{(0)}, \mathbf{m}^{(1)}, (A^{(0)}, e^{(0)}), (A^{(1)}, e^{(1)})$ with $\{m_i^{(0)}\}_{i \in \mathcal{D}} = \{m_i^{(1)}\}_{i \in \mathcal{D}}$,

$$(A^{(0)})^{x+e^{(0)}} = g \prod_i h_i^{m_i^{(0)}} \neq 1, \text{ and } (A^{(1)})^{x+e^{(1)}} = g \prod_i h_i^{m_i^{(1)}} \neq 1. \quad (1)$$

Note in particular this implies $A^{(0)}, A^{(1)} \neq 1$. For $i \in \mathcal{D}$ let $m_i = m_i^0 = m_i^1$, let $B^{(b)} = g \prod_i h_i^{m_i^{(b)}}$, and let Expt_b refer to the experiment in which the honest prover using $\mathcal{D}, \mathbf{m}^{(b)}, A^{(b)}, e^{(b)}$ interacts with $\mathcal{A}$. First note that the initial messages $\{m_i\}_{i \in \mathcal{D}}, A', \bar{B}$ sent in $\text{Expt}_0, \text{Expt}_1$ are identically distributed, since $A', \bar{B}$ are independently uniform in $\mathbb{G} \setminus \{1\}$ in either case. Fixing some $A', \bar{B}$ sent in the initial message, with corresponding randomness $r_1^{(b)}, r_2^{(b)}$ in Expt_b , we have

$$\begin{aligned} (A')^{-e^{(b)}} \bar{B}^{r_2^{(b)}} &= \left((A^{(b)})^{-e^{(b)}} B^{(b)} \right)^{r_1^{(b)} r_2^{(b)}} && \text{(by definition of } r_1^{(b)}, r_2^{(b)}) \\ &= \left((A^{(b)})^x \right)^{r_1^{(b)} r_2^{(b)}} && \text{(by (1))} \\ &= (A')^x && \text{(by definition of } r_1^{(b)}, r_2^{(b)}). \end{aligned}$$

Thus, in both $\text{Expt}_0, \text{Expt}_1$, the prover proves knowledge of a representation of $(A')^x$ relative to A' and $\bar{B}$, along with a representation of $g \cdot \prod_{i \in \mathcal{D}} h_i^{m_i}$ relative to $\bar{B}$ and $\{h_i\}_{i \in \mathcal{D}}$ using the Okamoto-Schnorr Σ -protocol [8], which is itself known to be WI. This completes the proof. $\square$

Remark: We do not prove zero knowledge (ZK); note that prior proofs of ZK assume the simulator is able to obtain some additional information that depends on the key x used to issue a tag. We instead prove WI, which suffices for proving anonymity, without any additional assumptions.

References

- [1] M.H. Au, W. Susilo, and Y. Mu. Constant-size dynamic k -TAA. SCN 2006.
- [2] A. Barki, S. Brunet, N. Desmoulins, and J. Traoré. Improved algebraic MACs and practical keyed-verification anonymous credentials. SAC 2016.
- [3] D. Boneh, X. Boyen, and H. Shacham. Short group signatures. Crypto 2004.
- [4] J. Camenisch, M. Drijver, and A. Lehmann. Anonymous Attestation Using the Strong Diffie Hellman Assumption Revisited. TRUST 2016. Available at <https://ia.cr/2016/663>.
- [5] J. Camenisch and A. Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. Crypto 2004.
- [6] J. Katz and Y. Lindell. Introduction to Modern Cryptography (third edition). Chapman & Hall/CRS Press, 2021.
- [7] T. Looker, V. Kalos, A. Whitehead, and M. Lodder. The BBS Signature Scheme. Internet Draft draft-irtf-cfrg-bbs-signatures-07, Internet Engineering Task Force, September, 2024. Available at <https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/07>. See also <https://github.com/decentralized-identity/bbs-signature>.
- [8] T. Okamoto. Provably Secure and Practical Identification Schemes and Corresponding Signature Schemes. Crypto '92.
- [9] M. Orrù. Revisiting Keyed-Verification Anonymous Credentials. Available at <https://eprint.iacr.org/2024/1552>.
- [10] S. Tessaro and C. Zhu. Revisiting BBS Signatures. Eurocrypt 2023. Available at <https://ia.cr/2023/275>.